

Anvil: executable verification of LLM-generated HPC operational artifacts

Antonio Rotundo

antonio.rotundo2@studio.unibo.it

August 7, 2026

Abstract

Assistants that write SLURM job scripts for supercomputer users are evaluated with similarity metrics, because no benchmark verifies whether the artifacts they produce actually work. Anvil verifies them by execution against a declared reference cluster: five levels covering syntax, submittability, execution, effective resource fit and safety, bracketed by an oracle that must score 1.0 and a deliberately faulty model that must score 0.0. On eight from-scratch tasks and 220 induced repair tasks we measure four models across three families, and report four results, two of them deflationary about the benchmark’s own machinery. Submittability is not ordered by model size: on the from-scratch set a 3B model of one family and a 12B of another lead it while a 7B trails both, and within one family the level falls as size rises. Real job submission changes exactly one verdict in 3984 artifacts against a **bash** sandbox, so the most expensive component of the harness is nearly redundant on this task set. Retrieved documentation costs the smaller model 28 points of resource fit whatever the text says, while the larger one is unaffected by irrelevant text and loses 6 points to the one document that addresses the level it concerns. A task built so that GNU coreutils and **utils** must judge it differently is failed by all three models for reasons that precede the difference. We also report, rather than conceal, three measurement faults found and corrected during the work, one of which had already been published.

1 Introduction

A wrong `--mem` does not look wrong. It looks plausible, and the job dies at hour six.

Operational HPC artifacts, batch scripts and container recipes, sit in a gap. Execution-based evaluation is standard for program synthesis and increasingly common for infrastructure code, yet the assistants appearing for HPC user support are still scored by how closely their output resembles a reference answer. Resemblance is the wrong instrument here for a specific reason: the failures that matter are invisible in the text. A directive placed after the first command is silently ignored by the scheduler. An omitted `--ntasks` is not an absent request but a request for a different number. A memory figure one factor too small parses, submits, and runs until it does not.

Unlike cloud infrastructure code, where a plan cannot be executed cheaply and evaluation falls back on graph edit distance or model judges, these artifacts are directly checkable. `sbatch --test-only` costs milliseconds, the job really runs, and the ground truth is objective. The empty slot is not a hard problem; it is an unoccupied one.

This paper reports what the resulting instrument says about four models and, with more care, what it says about itself.

2 Related work

The works below were selected from a sweep of fourteen queries against OpenAlex, Crossref, DBLP and arXiv, run by `scripts/litsweep.py` and returning 761 unique records. Its full output is committed with this manuscript, so the selection can be audited rather than taken on trust.

Assistants for HPC users, and how they are judged. Support assistants for super-computer users are being built and deployed: a chatbot for autonomous user support [6], a science-gateway assistant whose outcomes were reported from a national laboratory [1], and a Slurm-native serving infrastructure for running such models on the cluster itself [3]. Zheng et al. [10] compare several of these approaches. The task predates the models: the NERSC job script generator [8] produced batch scripts from a form, with correctness guaranteed by construction rather than measured. What is missing across this line is an execution-grounded way to say whether the artifacts these systems emit would run, which is the gap this work addresses.

Language models for HPC code. A parallel line targets the source code rather than the operational artifacts around it: modelling parallel programs [5], zero-shot parallelization guided by graph neural networks [4], and MPI code generation for HPC workloads [9]. These evaluate a kernel that compiles and runs. A batch script is judged by a scheduler instead, and fails in ways a compiler has no opinion about: a directive after the first command is accepted and ignored, and an omitted one is not an absent request but a different one.

Execution-grounded evaluation elsewhere. Outside HPC, execution is the norm. Chen et al. [2] established the unbiased $\text{pass}@k$ estimator this work uses unchanged. Infrastructure-as-code is the closest neighbour and shows the constraint: unable to execute a plan cheaply, Santos Vargas et al. [7] benchmark generated Terraform through static analysis. HPC operational artifacts do not share that constraint. `sbatch --test-only` costs milliseconds and the job really runs, which is why the ground truth here can be objective where theirs cannot.

3 The benchmark

T1, writing from scratch. Eight tasks, each a support request in ordinary words paired with a machine-readable specification: node and task counts, walltime and memory bounds, GPUs, directives the task demands explicitly, and strings the script must print when it runs.

T2, diagnosing and repairing. 220 tasks derived mechanically from the T1 canonical solutions by seven fault injectors, anchored to classes observed on a real model rather than invented: an omitted directive whose SLURM default hides it, a misplaced directive, prose leaking into a value, an absent no-default directive, a script stripped of every directive, a payload that no longer matches its own specification, and a value the scheduler rejects. Induction keeps only the variants that actually fail verification, since an injector producing an accidentally valid script is a bug in the injector and not a fault worth teaching a model to repair. Repairs are graded by the same verifier as T1, with no partial credit for being closer to correct.

4 Method

Five levels. The strict metric requires all of them.

syntax a shebang, `bash -n`, and no directive after the first command.

Table 1: T1, writing a job script from scratch. pass@1 per level. Safety is 1.000 ± 0.000 for every model and is omitted.

model	syntax	submit.	functional	res. fit	strict
Qwen2.5-Coder 1.5B	0.575 ± 0.025	0.842 ± 0.013	0.533 ± 0.037	0.442 ± 0.013	0.308 ± 0.025
Granite 4.1 3B	1.000 ± 0.000	0.875 ± 0.000	0.842 ± 0.037	0.625 ± 0.000	0.500 ± 0.000
Gemma 4 12B	0.875 ± 0.000	0.867 ± 0.013	0.875 ± 0.000	0.917 ± 0.050	0.658 ± 0.062
Qwen2.5-Coder 7B	1.000 ± 0.000	0.792 ± 0.025	0.875 ± 0.000	1.000 ± 0.000	0.667 ± 0.025

submittability sbatch --test-only accepts it.

functional it runs and exits zero.

resource fit the *effective* request matches the specification, with SLURM’s documented defaults applied.

safety destructive-pattern probes, which gate execution.

A skipped level is never a passed one. Without a scheduler, submittability is skipped and scored as not passed. One refinement was forced by measurement: a level skipped because *this machine* lacks a capability is pass-through, since every artifact is skipped alike, whereas a level skipped because *this artifact* cannot be judged anywhere is a failure. Before the distinction existed, ten repairs of a dependency task passed strict by being unjudgeable.

Effective requests, not string presence. A serial script omitting --nodes still requests one node and is correct. An early version checked for the string and failed scripts the scheduler accepts, which is precisely the surface-form matching this work exists to replace.

Both bounds shipped. An oracle returning canonical solutions must score 1.0 or the benchmark is defective; a broken model returning faulty artifacts must score 0.0 strict, and its sampling is arranged so the destructive flavour is always drawn, because a guard that never fires is decoration.

A declared reference cluster. Four nodes, sixteen cores and one GPU each, in a container running a live slurmctld, so that scores do not depend on the hardware of whoever runs the benchmark. Two preflights protect it: a canary that distinguishes a failing scheduler from a failing model, and a topology canary that refuses to score submittability when the scheduler in front of it is not the declared cluster.

Generation and verification are separate. Generation needs an accelerator; faithful grading needs the scheduler and GNU coreutils. Generations travel as JSONL carrying the digest of the task file, and the verifier refuses any whose digest does not match.

5 Results

Three seeds, $n = 5$ samples per task, 4-bit quantization on an RTX 3060, graded inside the container under the bash executor. The $\pm$ is half the range across seeds. It is not a confidence interval, and no significance is claimed anywhere in this paper.

Table 2: T2, diagnosing and repairing an induced fault. pass@1 per level. Safety is 1.000 ± 0.000 for every model and is omitted.

model	syntax	submit.	functional	res. fit	strict
Qwen2.5-Coder 1.5B	0.792 ± 0.016	0.886 ± 0.005	0.664 ± 0.005	0.412 ± 0.014	0.291 ± 0.000
Granite 4.1 3B	0.862 ± 0.002	1.000 ± 0.000	0.750 ± 0.000	0.753 ± 0.009	0.661 ± 0.009
Gemma 4 12B	1.000 ± 0.000	0.876 ± 0.002	0.885 ± 0.002	0.938 ± 0.002	0.732 ± 0.000
Qwen2.5-Coder 7B	0.983 ± 0.002	0.977 ± 0.000	0.870 ± 0.002	0.965 ± 0.007	0.824 ± 0.002

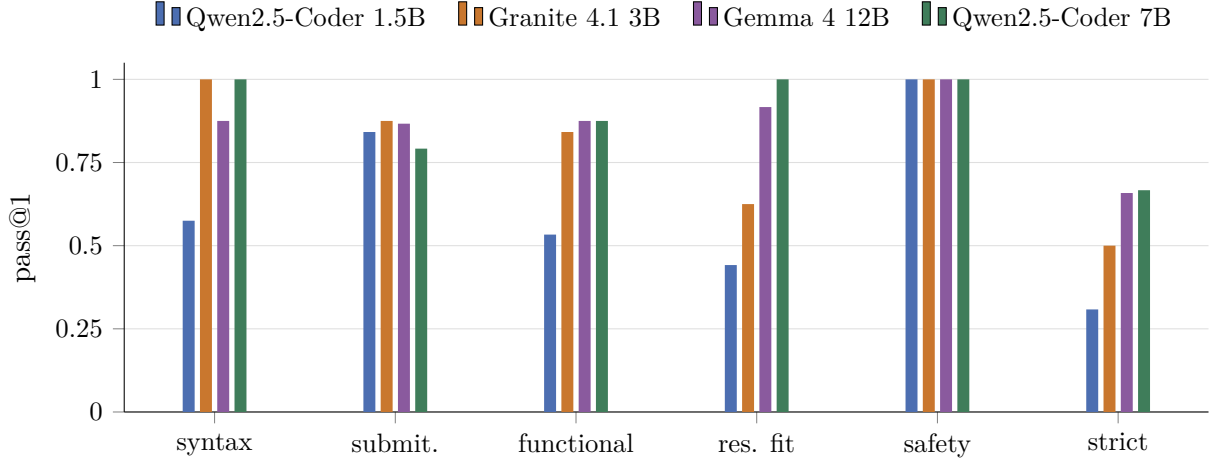


Figure 1: T1 per level. Every level rises with capability except submittability, where the 3B and the 12B lead and the 7B trails all three.

5.1 Submittability is not ordered by size

On T1 the level reads 0.875 for a 3B model, 0.867 for a 12B, 0.842 for a 1.5B and 0.792 for a 7B. The largest model in the set and the smallest of another family sit together at the top and the 7B is alone at the bottom, while within the Qwen family the level falls as size rises and every other level improves, which Figure 1 shows as the one bar group running against the others. Two families of three are above Qwen across a range from 3B to 12B, so the shortfall is not one model’s quirk.

T2 does not repeat that ordering, and it would be overclaiming to say otherwise: there every model sits between 0.876 and 1.000, which leaves little room to separate them, and the family that leads T1 is last.

The families also fail the level differently. One invents partition names, which a different cluster might genuinely have; another invents an option SLURM does not have, which no cluster has. The level measures how much a model volunteers that nothing asked for, and whose vocabulary it reaches for when it does.

5.2 Real submission is nearly redundant here

Grading the same generations twice inside the same image, once through a `bash` sandbox and once through real `sbatch`, exactly one artifact of 3984 changes verdict, and it changes in favour of real submission, which accepts a script the sandbox wrongly rejects. Every script real submission stopped was already failing another level.

This is a statement about these eight tasks and not about execution. A task built to need it does need it: a memory under-request that only cgroup enforcement can observe is invisible to every static level and to the sandbox, and there the verdict does change.

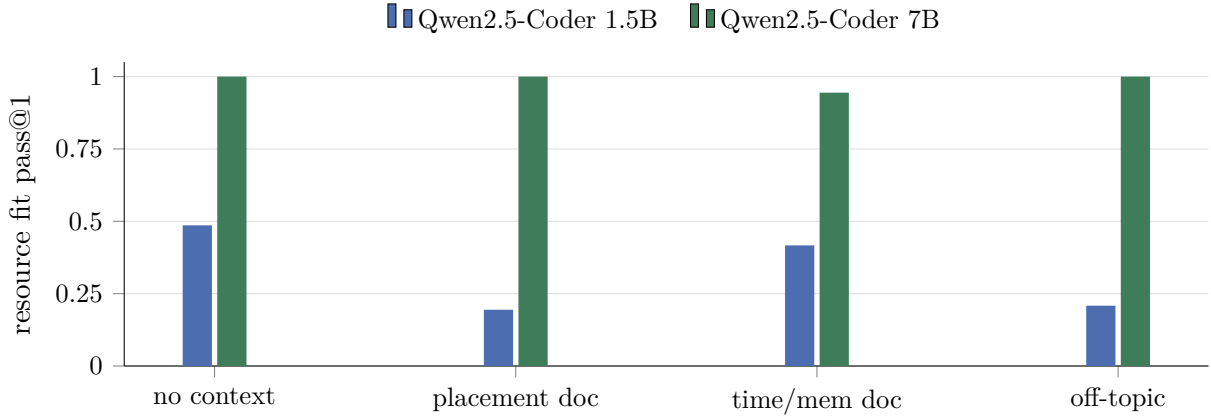


Figure 2: Resource fit under four retrieval conditions. The small model pays a toll for any attached text, including a length-matched passage on an unrelated subject; the large model pays none, and loses ground only to the document about the two directives this level checks.

5.3 Retrieval costs, and the sign depends on prior knowledge

Seven conditions across two sizes, four of them shared and plotted in Figure 2. Any attached text costs the 1.5B about 28 points of resource fit, whether it is SLURM documentation or a passage about coastal tides cut to the same length. The 7B pays no such toll: two unrelated off-topic controls leave it on its zero-shot values to the digit. The single corpus document addressing `--time` and `--mem` is worth +21 points to the model that does not know those directives have no default, and -6 to the one that does.

Capacity decides whether a model can ignore what does not concern it, and relevance decides the sign of what it cannot ignore.

5.4 A toolchain difference no model reaches

GNU coreutils 9.4 and `utils` 0.8.0 agree on 91 of 101 probed invocations. Three of the disagreements are behavioural and need no misconfigured locale: character counting and column expansion on a non-ASCII payload, and SI number formatting. A task built to sit in that corner is judged differently by the two implementations, which a guard asserts.

Asked to solve it, three models produced 45 samples: none pinned a locale, and none diverged. Each fails on both implementations for a reason that precedes the difference, one of them by emitting escape sequences as literal text and so removing the non-ASCII content the question was about.

6 What went wrong while measuring

Reported because the corrections are load-bearing.

A published multi-seed table had been graded against the experiment machine’s own scheduler, which has one node, no GPUs, and rejects three of the eight canonical solutions. Submitability and strict were wrong by up to 36 points, while syntax, functional and resource fit were unchanged to the digit, which is the signature that identified the fault. The topology canary exists because of it, and fired on that same machine the first time a model was run after it landed.

The same defect was later found in the retrieval ablation, whose two scheduler-dependent columns were withdrawn and remeasured. Correcting them inverted an arm ordering: a single-seed pilot had been right, and the larger run that appeared to correct it had been wrong. The cause was the grading environment, not the seeds.

Three mechanisms proposed for the retrieval effect were each refuted by the experiment run to test them. The fourth was found only by counting per sample instead of per problem.

7 Threats to validity

Eight T1 tasks, which makes each one worth 0.125 of every T1 figure. Three seeds, so the ranges are spread rather than intervals. Three model families at four sizes, from 1.5B to 12B, all quantized to 4 bit. One reference topology, declared by this work rather than borrowed from a centre in production. All prompts written by one author, in one language and one style. No result here has been replicated independently.

8 Availability

Code, tasks, canonical solutions, the digest manifest identifying the dataset version, the generated leaderboard and every figure above are at <https://github.com/antoniorotundo2/anvil> under MIT. The command `anvil check` applies the verifier to a single artifact, with or without a site policy, for readers who want the instrument rather than the measurements.

References

- [1] Brandon S. Biggs and Kaylee Dalton. Outcomes of HPC user support using a science gateway AI assistant. Technical report, Idaho National Laboratory, 2024.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.
- [3] Ali Doosthosseini, Jonathan Decker, Hendrik Nolte, and Julian Kunkel. Chat AI: A seamless slurm-native solution for HPC-based services. *arXiv preprint arXiv:2407.00110*, 2024. doi: 10.48550/arXiv.2407.00110.
- [4] Quazi Ishtiaque Mahmud, Ali TehraniJamsaz, Hung Phan, Le Chen, and Mihai Capota. AutoParLLM: GNN-guided context generation for zero-shot code parallelization using LLMs. *arXiv preprint arXiv:2310.04047*, 2023. doi: 10.48550/arXiv.2310.04047.
- [5] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhatele. HPC-coder: Modeling parallel programs using large language models. *arXiv preprint arXiv:2306.17281*, 2023. doi: 10.48550/arXiv.2306.17281.
- [6] Rajesh Pasupuleti, Ravi Vadapalli, Christopher Mader, and Subhan Shakoore. LLM chatbot for autonomous user support in high-performance computing. In *International Conference on Machine Learning and Applications (ICMLA)*, 2025. doi: 10.1109/icmla66185.2025.00226.
- [7] Francis Luis Santos Vargas, Rodrigo Brandão Mansilha, and Diego Kreutz. Can language models generate secure Terraform code? a security-focused benchmark using static analysis. In *Workshop on Cloud Computing Research*, 2026. doi: 10.5753/pesquisanuvem.2026.23954.
- [8] Robin Shao, Thorsten Kruth, and Zhengji Zhao. NERSC job script generator. In *IEEE/ACM International Workshop on HPC User Support Tools (HUST)*, 2022. doi: 10.1109/hust56722.2022.00010.

- [9] Pedro Valero-Lara, Aaron Young, Thomas Naughton III, Christian Engelmann, and Al Geist. ChatMPI: LLM-driven MPI code generation for HPC workloads. In *Supercomputing Asia*, 2026. doi: 10.1145/3773656.3773659.
- [10] Mingkai Zheng, Fangru Linghu, Sikan Li, Marty Charles Kandes, and Niall Gaffney. A comparative study on LLM-enabled HPC user support. In *Practice and Experience in Advanced Research Computing (PEARC)*, 2026. doi: 10.1145/3785462.3815796.